

مستندات پروژه Netease - Asset Management System

 ساختار کلی پروژه

```
~/main_app/
├── venv/                # (ادست نزنید) محیط مجازی Python
├── netease/             # 🎯 پروژه اصلی
│   ├── app/            # کد اصلی برنامه
│   │   ├── __init__.py
│   │   └── main.py      # 🔥 نقطه ورود اصلی
│   └── core/           # 🧠 هسته مشترک سیستم
│       ├── config.py    # تنظیمات (DATABASE_URL, SECRET_KEY, ...)
│       ├── database.py  # اتصال به PostgreSQL
│       └── security.py  # امنیت (Hash, JWT Token)
│   ├── models/         # 📄 مدل‌های دیتابیس (جداول)
│       ├── __init__.py
│       ├── user.py      # جدول users
│       └── login_log.py # جدول login_logs
│   ├── schemas/        # 📝 ساختار داده‌ها (Input/Output)
│       ├── __init__.py
│       ├── auth.py      # UserLogin, Token
│       ├── user.py      # UserCreate, UserUpdate, UserResponse
│       └── log.py       # LoginLogResponse
│   ├── modules/        # 📦 ماژول‌های اصلی (هر کدام مستقل)
│       ├── auth/        # 🛡️ ماژول احراز هویت
│       │   ├── router.py # API: POST /auth/login
│       │   └── service.py # منطق: authenticate_user()
│       ├── logs/        # 🇮🇷 ماژول لاگ‌ها
│       │   └── router.py # API: GET /api/logs/...
│       ├── users/       # 👤 ماژول مدیریت کاربران (بعدی)
│       │   ├── router.py # API: CRUD کاربران
│       │   └── service.py # منطق: create, update, delete
│       └── utils/       # 🛠️ ابزارهای کمکی
│   ├── scripts/         # 📜 اسکریپت‌های کمکی
│       └── create_users.py # ساخت کاربران اولیه
│   ├── tests/           # 📱 تست‌ها (آینده)
│   └── alembic/          # 🔄 Migration (آینده) های دیتابیس
├── .env                 # 🗝️ (SECRET!) متغیرهای محیطی
├── requirements.txt      # 📦 لیست پکیج‌های Python
└── README.md            # 📖 راهنمای پروژه
```

🔥 فایل های کلیدی و وظایف آنها

1. `app/main.py` - ❤️ قلب برنامه

و ثبت تمام مازول ها FastAPI وظیفه: راه اندازی

:کی دستکاری می کنیم

- ✅ وقتی مازول جدید اضافه می کنیم
- ✅ عوض میشه CORS وقتی تنظیمات
- ❌ هیچ منطق کسبوکار اینجا ننویسید!

:ساختار

```
python

# 1. Import ها
from app.modules.auth import router as auth_router
from app.modules.logs import router as logs_router

# 2. ساخت App
app = FastAPI(title="...")

# 3. تنظیمات (CORS)
app.add_middleware(...)

# 4. ثبت Router ها
app.include_router(auth_router.router, prefix="/auth")
app.include_router(logs_router.router, prefix="/api/logs")

# 5. Endpoint های پایه
@app.get("/")
@app.get("/health")
```

`include_router` قانون: هر مازول جدید = یک خط

2. `app/core/` - 🧠 هسته سیستم

`config.py`

- **وظیفه:** نگهداری تنظیمات کلی
- **شامل:** DATABASE_URL, SECRET_KEY, CORS, ...
- **کی تغییر می‌ده:** نادر (فقط تنظیمات جدید)

database.py

- **PostgreSQL** **وظیفه:** اتصال به
- **شامل:** Engine, SessionLocal, get_db()
- **کی تغییر می‌ده:** تقریباً هیچوقت

security.py

- **امنیت و رمزنگاری** **وظیفه:**
- **شامل:** Hash password, JWT Token
- **کی تغییر می‌ده:** اضافه کردن توابع امنیتی جدید

قانون: هر چیزی که همه ماژول‌ها ازش استفاده می‌کنن، اینجا میاد

3. مدل‌های دیتابیس - `app/models/`

وظیفه: تعریف جداول دیتابیس

هر فایل = یک جدول:

- `user.py` → جدول `users`
- `login_log.py` → جدول `login_logs`

کی فایل جدید می‌سازیم:

- وقتی جدول جدید می‌خواهیم

مثال:

```
python
# app/models/asset.py (پنده)
class Asset(Base):
    __tablename__ = "assets"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    ...
```

کنید export `models/__init__.py` جدید، در `model` **قانون مهم:** بعد از اضافه کردن

python

```
from .user import User
from .login_log import LoginLog
from .asset import Asset # جدید

__all__ = ["User", "LoginLog", "Asset"]
```

4. ساختار داده‌ها - `app/schemas/`

(Validation) ها API وظیفه: تعریف ساختار ورودی/خروجی

انواع:

- **Input Schemas:** (مثل `UserLogin`) میاد React داده‌ای که از
- **Output Schemas:** (مثل `Token`) میره React داده‌ای که به

مثال:

```
python

# Input (Request)
class UserLogin(BaseModel):
    username: str
    password: str

# Output (Response)
class Token(BaseModel):
    access_token: str
    token_type: str
```

چرا لازمه:

-  Validation خودکار (Pydantic)
-  JSON تبدیل خودکار به
-  مستندسازی خودکار در `/docs`

5. ماژول‌های اصلی - `app/modules/`

!قانون طلایی: هر ماژول کاملاً مستقل است

ساختار استاندارد هر ماژول:

```
modules/example/
├── __init__.py      # Export router
├── router.py        # API Endpoints (FastAPI)
├── service.py       # Business Logic
├── schemas.py       # Schemas (اختیاری) مخصوص این مازول
└── utils.py        # توابع کمکی (اختیاری)
```

router.py - API Endpoints

```
python

from fastapi import APIRouter

router = APIRouter()

@router.get("/")
def get_items():
    # فقط دریافت و برگشت داده
    # بسپار service منطق رو به
    pass
```

فقط Router: قانون

- ✓ دریافت درخواست
- ✓ service فراخوانی
- ✓ برگشت پاسخ
- ✗ بدون منطق پیچیده!

service.py - Business Logic

```
python

class ExampleService:
    def __init__(self, db):
        self.db = db

    def get_all(self):
        # تمام منطق اینجاست
        return self.db.query(Model).all()
```

شامل Service: قانون

-  Query های دیتابیس
-  منطق کسبوکار
-  Validation های پیچیده
-  **FastAPI!** بدون وابستگی به

کامل Request جریان یک

1. React: `axios.post('/auth/login', {username, password})`
↓
2. FastAPI: `app.main.py` → routes request به auth module
↓
3. Router: `modules/auth/router.py` → `@router.post("/login")`
↓
4. Service: `modules/auth/service.py` → `authenticate_user()`
↓
5. Model: `models/user.py` → `db.query(User)`
↓
6. Database: PostgreSQL → `SELECT * FROM users WHERE username=...`
↓
7. Security: `core/security.py` → `verify_password()`, `create_token()`
↓
8. Schema: `schemas/auth.py` → Token (validation & serialize)
↓
9. Response: JSON به React برمی‌گردد

چک‌لیست ساخت مازول جدید

users برای ساخت مازول:

- ☐ 1. ساخت پوشه: `mkdir app/modules/users`
- ☐ 2. فایل `__init__.py`: `export router`
- ☐ 3. فایل `router.py`: تعریف API endpoints
- ☐ 4. فایل `service.py`: نوشتن منطق
- ☐ 5. Schema (اختیاری) در `app/schemas/`
- ☐ 6. Model (اختیاری) در `app/models/`
- ☐ 7. ثبت در `main.py`: `app.include_router(...)`
- ☐ 8. تست: `curl` و `/docs`

قوانین کلی

✅ DO (انجام بده):

- هر ماژول مستقل باشه
- router در API، service، منطق در
- Model ها در `models/`
- Schema ها در `schemas/`
- هر تغییر رو تست کن

❌ DON'T (انجام نده):

- router منطق پیچیده در
- Import (مگه) از ماژول دیگه
- (استفاده کن config از) کردن مقادیر Hard-code
- در یک فایل Schema و Model

دستورات مهم

اجرای برنامه

```
bash

cd ~/main_app/netease
source ../venv/bin/activate
python -m app.main
```

ما API مشاهده:

```
http://localhost:8000/docs
```

دسترسی به دیتابیس:

```
bash

psql -U netease -d netease_db
```

نکات مهم برای توسعه

1. قبل از شروع مازول جدید:

- Schema ها رو طراحی کن
- Model (جدول) لازمه؟
- Endpoint ها چی هستند؟

2. حین توسعه:

- رو جداگانه تست کن Endpoint هر
- استفاده کن `/docs` از
- debug لاگ بذار برای

3. بعد از اتمام:

- ها رو تست کن Endpoint تمام
- مستندات برای فرانت بنویس
- کن Git commit در

این مستندات رو نگه دار و هر بار که مازول جدید اضافه کردی، بهروزش کن 